IA

Inventory Analyzer

READ-ONLY FOLDER AND FILE INVENTORY

TECHNICAL DESIGN

Technical Design

Architecture, data model, scanning and calculation logic, storage, exports, security and testing

Document ID IA-TD-05 · **Version** 1.2 · **Date** 24-Oct-2026

Prepared by Yasin Jariwala

Audience Developers, reviewers and security teams

Document control

Document	Technical Design
Document ID	IA-TD-05
Version	1.2
Date	24-Oct-2026
Prepared by	Yasin Jariwala
Product	Inventory Analyzer v1.2
Audience	Developers, reviewers and security teams
Classification	Public
Source	github.com/yasin679/folder-file-inventory-analyzer · docs/technical-design.md

Version history

Version	Date	Author	Changes
1.0	24-Oct-2026	Yasin Jariwala	First release: read-only folder scan, dashboard, inventory, storage analysis, optional duplicates, Excel and CSV exports, scan history
1.1	24-Oct-2026	Yasin Jariwala	Folder Explorer, Where your files are, Show in folder tree, Folder Tree sheet
1.2	24-Oct-2026	Yasin Jariwala	Reopen saved scans and download their reports; seven numbered guides inside the app

Related documents

ID	Document
IA-UG-01	User Guide
IA-IG-02	Installation and Administration Guide
IA-QS-03	Quick Start
IA-FAQ-04	FAQ and Troubleshooting
IA-UAT-06	UAT Test Cases
IA-RN-07	Release Notes v1.2

Contents

1 Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Design principles 3
2 Architecture 4
2.1 Overview 4
2.2 Components 4
2.3 Technology choices 5
2.4 The walker interface 5
3 Data model 7
3.1 ScanRecord 7
3.2 FileRecord 7
3.3 FolderRecord 7
3.4 ScanError 8
3.5 DuplicateGroup 8
3.6 ScanConfig 8
4 Scanning and calculation rules 10
4.1 Aggregation 10
4.2 Duplicate detection (FR-08)
4.3 Progress and cancellation (FR-11)
5 Local storage (FR-13)
5.1 IndexedDB schema 11
5.2 Behaviour 11
6 Exports (FR-09, FR-10)
6.1 Report tables 12
6.2 Excel 12
6.3 CSV 12
6.4 Downloads 12
7 Security and privacy 13
8 Performance and limits 14
9 Testing 15
10 Build and delivery 16

1 Introduction

1.1 Purpose

This document describes the design of Inventory Analyzer v1.2: its architecture, components, data model, scanning and calculation rules, duplicate detection, local storage, exports, security controls and tests. It is the technical reference for developers, reviewers and security teams. The requirements it implements are in the PRD v3.0 (`docs/PRD and Master Prompt.docx`); requirement IDs such as FR-03 refer to it.

1.2 Scope

- **Input:** one folder chosen by the user, opened read-only through the browser.
- **Output:** an inventory of every file and folder, derived reports on screen, and Excel and CSV exports.
- **Runtime:** a single static web page. No server, database, account system or network calls.
- **Out of scope for v1.2:** comparing two scans, scanning cloud storage directly, changing or deleting files, and inventories larger than the browser's memory.

1.3 Design principles

Principle	How it is applied
Read-only	The folder picker is requested with `mode: 'read'`. No create, write, move, rename or delete API is called anywhere; a test fails if one is.
Local-first	All work happens in the browser tab. No request carries scan data; a test watches the network during scan and export.
No guessing	A folder that cannot be listed is `unknown`, never `empty`. A file that cannot be hashed is `unverified`, never `duplicate`. Unavailable data (created date, full path, hidden attribute) is not shown rather than estimated.
Complete accounting	Each file is counted once in its folder's direct totals and once in every ancestor's recursive totals. Export row counts are reconciled against the scan before a file is produced.
One source of truth	On-screen reports, the Excel workbook and the CSVs are built from the same table definitions.
Responsive	Scanning runs in a Web Worker; long loops yield regularly so cancel and progress messages are handled.
Minimal footprint	One HTML file, five runtime libraries bundled at build time, nothing fetched at run time.

2 Architecture

2.1 Overview



2.2 Components

Component	File(s)	Responsibility
Types	`src/core/types.ts`	Data model (section 3) and default configuration
Scanner	`src/core/scanner.ts`	`Scanner` class: folders, files, errors, direct and recursive aggregation, completeness, duplicate detection, progress; the two walkers; `runScan`
Controller	`src/core/controller.ts`	Starts the scan in an inline Web Worker, relays progress, ignores stale messages, falls back to the main thread
Worker	`src/worker/scan.worker.ts`	Worker entry: receives the handle or file list and configuration, runs `runScan`, posts progress and result
Analysis	`src/core/analysis.ts`	Extension summary, largest files and folders, empty folders, depth distribution, key findings, folder tree, top-level breakdown
History	`src/core/history.ts`	IndexedDB stores for scan summaries and full results; pruning to the newest 20

Component	File(s)	Responsibility
Sample	`src/core/sample.ts`	Deterministic synthetic "Shared Drive" handle tree for the demo
Tables	`src/export/tables.ts`	One definition per report: columns, kinds, widths, rows, expected row count
Excel	`src/export/xlsx.ts`	ExcelJS workbook: Executive Summary, formatting, sheet splitting, reconciliation
CSV	`src/export/csv.ts`	RFC 4180 CSV with BOM; ZIP bundle with fflate
Sanitize	`src/export/sanitize.ts`	Formula-injection neutralization
UI	`src/ui/App.tsx`, `screens.tsx`, `explorer.tsx`, `help.tsx`, `components.tsx`	Screens, navigation, downloads, history actions, embedded guides
Generated docs	`src/generated/docs.ts`	Guides, screenshots and PDFs embedded by `scripts/embed-docs.mjs`

2.3 Technology choices

Choice	Version	Reason
React + TypeScript	18.3 / 5.6	Typed UI with a large testing ecosystem
Vite + vite-plugin-singlefile	5.4	Fast builds; the single-file build inlines all scripts, styles and the worker into one HTML file
File System Access API	Browser	Read-only directory handles with per-item errors and empty-folder visibility
`webkitdirectory` input	Browser	Universal fallback for Firefox and Safari
Web Crypto `SHA-256`	Browser	Fast, native content hashing with no library
ExcelJS	4.4	Real `.xlsx` with styles, frozen panes, filters and number formats
fflate	0.8	Small, fast ZIP for the CSV bundle
Recharts	2.15	Dashboard and depth charts
marked	12.0	Renders the embedded Markdown guides
IndexedDB	Browser	Local scan history large enough for full results

2.4 The walker interface

Both access modes feed the same `Scanner`, so every rule is identical in both:

Walker	Input	Behaviour
`walkDirectoryHandle`	`FileSystemDirectoryHandle` (or the sample tree)	Depth-first over `entries()`. Each folder is added before its children. A failure listing a folder records `enumeration-failed` / `permission-denied`, marks the folder `unknown` and its ancestors incomplete. A failure reading one file records the error and keeps the file with `metadata-error`.
`walkFileList`	`FileList` from `<input webkitdirectory>`	Builds folders from each file's `webkitRelativePath`. The total number of files is known in advance, so a percentage is shown. Empty folders cannot be seen.

Dot-prefixed names are skipped (and counted as skipped) when `includeDotItems` is false. The scanner yields to the event loop regularly and checks the cancel flag.

3 Data model

All records live in memory as plain objects; a `ScanResult` holds them all. Field names follow PRD §10. Sizes are bytes; times are milliseconds since 1970 (UTC).

3.1 ScanRecord

Field	Meaning
`scan_id`	`scan-YYYYMMDDHHMMSS-xxxxxxxx`
`root_label`	Name of the selected folder
`access_mode`	`file-system-access` or `directory-input`
`started_at`, `completed_at`	Scan start and end
`status`	`running`, `completed`, `completed-with-errors`, `cancelled`, `failed`
`configuration`	The `ScanConfig` used (section 3.6)
`files_discovered`, `folders_discovered`	Totals; folders include the root
`total_accessible_size_bytes`	Sum of readable file sizes
`error_count`, `skipped_count`	Errors recorded; items skipped by configuration
`duplicate_groups`, `redundant_bytes`	Duplicate totals (0 when detection is off)

3.2 FileRecord

Field	Meaning
`file_id`, `folder_id`	Index in `files`; owning folder
`file_name`, `relative_path`, `parent_path`	Name; path from the root including the root's name; the folder's path
`extension`	Lower-case, without the dot; empty when none
`size_bytes`, `modified_at`	Logical size; last-modified time when exposed
`dot_prefixed`	Name starts with "."
`scan_status`	`ok` or `metadata-error`
`duplicate_status`, `duplicate_group`, `content_hash`	`not-checked`, `unique`, `duplicate` or `unverified`; group; SHA-256 hex

3.3 FolderRecord

Field	Meaning
`folder_id`, `parent_folder_id`	Index in `folders`; parent (null for the root)

Field	Meaning
`folder_name`, `relative_path`, `parent_path`, `depth`	Root is depth 0
`direct_file_count`, `recursive_file_count`	Files directly inside; files at any depth
`direct_folder_count`, `recursive_folder_count`	Subfolders directly inside; at any depth
`direct_size_bytes`, `recursive_size_bytes`	Size of direct files; of all files at any depth
`empty_status`	`has-files`, `subfolders-only`, `no-files-only-subfolders`, `empty` or `unknown`
`scan_completeness`	`complete` or `incomplete`

3.4 ScanError

Field	Meaning
`error_id`, `relative_path`, `item_type`	Item and whether it is a file, folder or the whole scan
`error_category`	`permission-denied`, `not-found`, `not-readable`, `enumeration-failed`, `metadata-unavailable`, `hash-failed`, `hash-skipped-too-large`, `cancelled`, `unknown`
`message`, `timestamp`	Browser message and time

3.5 DuplicateGroup

Field	Meaning
`group_id`, `content_hash`	Group number; shared SHA-256
`size_bytes`, `copy_count`	Size of each copy; number of copies
`redundant_bytes`	`size_bytes × (copy_count − 1)`
`file_ids`	Members

3.6 ScanConfig

Field	Default	Meaning
`includeDotItems`	true	Include names starting with "."
`detectDuplicates`	false	Hash same-size files with SHA-256
`largeFileThresholdBytes`	100 MiB	Large-file list and key finding
`topN`	25	Length of largest-items lists

Field	Default	Meaning
`maxHashBytes`	1 GiB	Files above this are `unverified`
`keepResults`	true	Store full results in history

4 Scanning and calculation rules

4.1 Aggregation

Rule	Detail
Direct vs recursive	Direct = immediate children. Recursive = all included descendants. When a file is added, its size and count go to its folder's direct totals and to the recursive totals of the folder and every ancestor.
Counted once	Each file belongs to exactly one folder. The root's recursive file count equals `files_discovered`.
Folders	`folders_discovered` includes the root. Recursive folder counts exclude the folder itself.
Empty status	`empty`: no files and no subfolders. `no-files-only-subfolders`: no files anywhere below, but subfolders. `subfolders-only`: no direct files, files deeper. `has-files`: direct files. `unknown`: the folder could not be listed.
Completeness	A folder is `incomplete` if it, or anything below it, could not be fully read, or the scan was cancelled.
Status	`completed` without errors; `completed-with-errors` with any error; `cancelled` when stopped; `failed` when the scan threw before producing a result.

4.2 Duplicate detection (FR-08)

1. Group non-empty files by `size_bytes`. A file with a unique size is `unique` without being read.
2. For each size shared by two or more files, read each file (`Blob.arrayBuffer`, or `FileReader` where unavailable) and compute SHA-256 with Web Crypto. Files above `maxHashBytes` are `unverified` with `hash-skipped-too-large`.
3. Files with equal hashes form a group (`duplicate`); a hash shared by nobody else is `unique`. A read failure is `unverified` with `hash-failed`.
4. Groups are numbered by redundant bytes, largest first. Contents are discarded after hashing.

Zero-byte files are `not-checked`. Names are never compared.

4.3 Progress and cancellation (FR-11)

- Progress is emitted at most every 120 ms with phase, counts, bytes, errors, elapsed time and current path.
- `percent` is set only when a real denominator exists: upload mode while enumerating (files known in advance) and while hashing. Otherwise the bar is indeterminate.
- `etaMs` is set only after 10% and 2 seconds, so early estimates are not shown.
- Cancel sets a flag checked between items; the scanner records a `cancelled` error and finalizes the partial result, which is labelled partial everywhere.
- Each scan has an ID; the controller ignores progress or results from an older scan, so a stale worker message can never overwrite a newer scan.
- If the worker cannot start (for example blocked by the page context) or fails before producing a result, the controller runs the same code on the main thread and the UI says so.

5 Local storage (FR-13)

5.1 IndexedDB schema

Database `folder-inventory-analyzer`, version 2.

Store	Key	Index	Contents
`scans`	`scan_id`	—	One `ScanRecord` per scan (the summary row)
`results`	`scan_id`	`started_at`	`{ scan_id, started_at, result }`: the full `ScanResult`

The upgrade from version 1 (summaries only) creates `results` and keeps every existing summary.

5.2 Behaviour

Operation	Detail
Save	After a successful or partial real scan, the summary is saved; if `keepResults` is on, the full result too. Sample runs are not saved.
Pruning	After each result is saved, results beyond the newest 20 (by `started_at`) are deleted in the same transaction. Their summaries stay ("Summary only").
Open	Loads the result and shows it exactly as a fresh scan, labelled **Saved scan**.
Download from history	Loads the result and runs the normal export without changing the open scan.
Delete / clear	Removes from both stores in one transaction.
Errors	Quota errors, a blocked upgrade (another tab on an older version) and an unavailable IndexedDB each produce a specific message; scanning and exports keep working.

Settings (except `detectDuplicates`) are kept in `localStorage`.

6 Exports (FR-09, FR-10)

6.1 Report tables

`buildTables(result)` returns the report tables in order: Folder Tree, File Inventory, Folder Summary, Extension Summary, Largest Files, Largest Folders, Duplicate Files (only when detection was on), Empty Folders and Scan Errors. Each table declares its columns (header, kind `text` / `int` / `date` / `pct`, width), its rows and its **expected row count** derived independently from the scan (for example `files.length + folders.length` for the Folder Tree).

6.2 Excel

Feature	Detail
Sheets	Executive Summary first (summary rows and key findings), then one sheet per table
Formatting	Bold header row, frozen header, auto-filter, column widths, `#,##0` for integers, date format for dates; Folder Tree indented by level with folders in bold; totals row on summary tables
Values	Text cells are written as values, never formulas, so file names starting with `=` are safe
Sheet splitting	Tables longer than 1,048,575 data rows continue on "Name (2)", "Name (3)", each with headers
Reconciliation	Before the file is produced, the rows written per table are compared with the expected counts; any mismatch aborts the export with an error and no file

6.3 CSV

Feature	Detail
Format	RFC 4180, CRLF line endings, UTF-8 with byte-order mark (so Excel detects UTF-8)
Escaping	Fields containing `"`, `,` or line breaks are quoted, quotes doubled
Injection	Strings beginning with `=`, `+`, `-`, `@`, tab or carriage return are prefixed with `'`
Names	`<folder>_<report>.csv`; the ZIP is `<folder>_inventory_<date>_csv.zip`
Reconciliation	Each CSV checks its row count against the expected count

6.4 Downloads

The app saves files through the host's download capability when it runs as a claude.ai artifact (each save is confirmed by the user, and a declined save is reported as "Save cancelled"), and through a standard `<a download>` link everywhere else.

7 Security and privacy

Threat or concern	Control
Accidental change to user files	Read-only picker mode; no write APIs used; test S25 asserts that only read mode is requested and no write method is called
Data leaving the PC	No network code paths; test S24 records every request during scan and export and fails on any non-origin request
Malicious file names in reports	CSV formula neutralization; Excel values never written as formulas; HTML output escaped by React
Exposure of disk layout	Only relative paths exist; browsers never expose absolute paths
Sensitive names in browser storage	Stored only on the user's PC; can be turned off (`keepResults`) and cleared from two screens
Third-party code	Five bundled libraries, pinned in `package-lock.json`; no CDN or run-time download
Embedded guides	Bundled Markdown written by the project, not user input

8 Performance and limits

Item	Result or limit
Sample folder (318 files)	Scanned in under 100 ms
Large test (26,160 files, 22,000 folders)	Scanned in seconds; reopened from history after a page reload in 0.4 s; Excel rebuilt from history in 6.2 s
Memory	The whole `ScanResult` is held in memory; a few hundred thousand files is the practical limit on a typical laptop
Tables	Paginated (25 to 250 rows) to keep the page fast
Folder Explorer	Shows the first 200 files of a folder until **Show all files** is clicked
Excel	1,048,576 rows per sheet; larger tables are split

9 Testing

The app was built test-first: each user scenario (S01 to S44, listed in `tests/SCENARIOS.md`) was written as a failing test before the code was changed.

Suite	Tool	Tests	Covers
Unit	Vitest	42	Scanner rules, walkers, errors, cancellation, stale progress, worker fallback, duplicates, folder tree, history (including the v1 to v2 upgrade and pruning), Excel and CSV content
UI scenarios	Vitest, jsdom, Testing Library, user-event	47	Every scenario through the real React app with clicks and typing; fake-indexeddb for history
End-to-end	Playwright, Chromium	9	Production build: real folder upload in a worker, picker path, Excel and ZIP reopened and reconciled, no network, phone layout, keyboard-only use, Folder Explorer, 26,000-file scan reopened after reload, Storage Analysis controls visible

Total: 98 tests, all passing; about 97% of statements covered. Writing tests first found 9 defects before release; they are listed in the Release Notes (IA-RN-07). The UAT cases for people are in IA-UAT-06.

10 Build and delivery

Step	Detail
Embed guides	`scripts/embed-docs.mjs` reads `docs/*.md`, inlines the screenshots from `docs/images/` and the PDFs from `docs/pdf/` (base64) into `src/generated/docs.ts`
Type check	`tsc --noEmit`
Bundle	`vite build --mode single` with vite-plugin-singlefile; the worker is imported with `?worker&inline` so it is inside the same file
Post-process	`scripts/escape-fffd.mjs` replaces raw U+FFFD characters with escapes, because some hosts reject them
Output	`dist-single/index.html`, copied to `Inventory Analyzer.html`
Guides	`docs/build/capture_screenshots.mjs` (Playwright) and `docs/build/build_pdfs.py` (Markdown, Chromium, pypdf)